Основы глубинного обучения

Лекция 6

Архитектуры свёрточных сетей

Евгений Соколов

esokolov@hse.ru

НИУ ВШЭ, 2026

Дисперсия после dropout

- Выход нейрона с inverted dropout: $m \cdot \frac{h}{q}$
  - h — выход нейрона до дропаута
  - $m \sim \text{Bernoulli}(q)$ — маска, $q = 1 - p$ — вероятность оставить нейрон
  - $\mathbb{E} = h$, но $\mathbb{D} = h^2 \cdot \frac{p}{q}$
- Точно скорректировать нельзя: следующий слой видит сумму $s = \frac{\sum_j w_j m_j h_j}{q}$, её дисперсия $\mathbb{D}s = \frac{p}{q} \sum_j w_j{}^2 h_j{}^2$ зависит от конкретных h и w — единого множителя нет; совпасть может только среднее
- И не нужно: шум работает как регуляризатор, следующий слой на обучении подстраивается под шумный масштаб
- Где это мешает: BatchNorm после дропаута запоминает дисперсию с шумом, на тесте она другая — variance shift (Li et al., 2018). Отсюда правило: dropout после BatchNorm или только в последних слоях
- Вариант с подобранной дисперсией — Gaussian dropout (Srivastava et al., 2014): умножать на шум $\mathrm{N}\left(1, \frac{p}{q}\right)$

https://arxiv.org/abs/1801.05134

Ещё раз про γ и β в BatchNorm

- Обычные обучаемые параметры — по паре на каждый нейрон (канал) каждого BatchNorm-слоя
- Стартуют с $\gamma = 1$, $\beta = 0$ (нормированный выход как есть) и учатся градиентным спуском вместе с весами
- Если перед ReLU: β сдвигает распределение и тем самым задаёт, какая доля входов обнулится
- Если перед сигмоидой — насколько заходим в нелинейную область

В чём польза от BatchNorm?

How Does Batch Normalization Help Optimization?

Shibani Santurkar* MIT shibani@mit.edu

Dimitris Tsipras* MIT tsipras@mit.edu

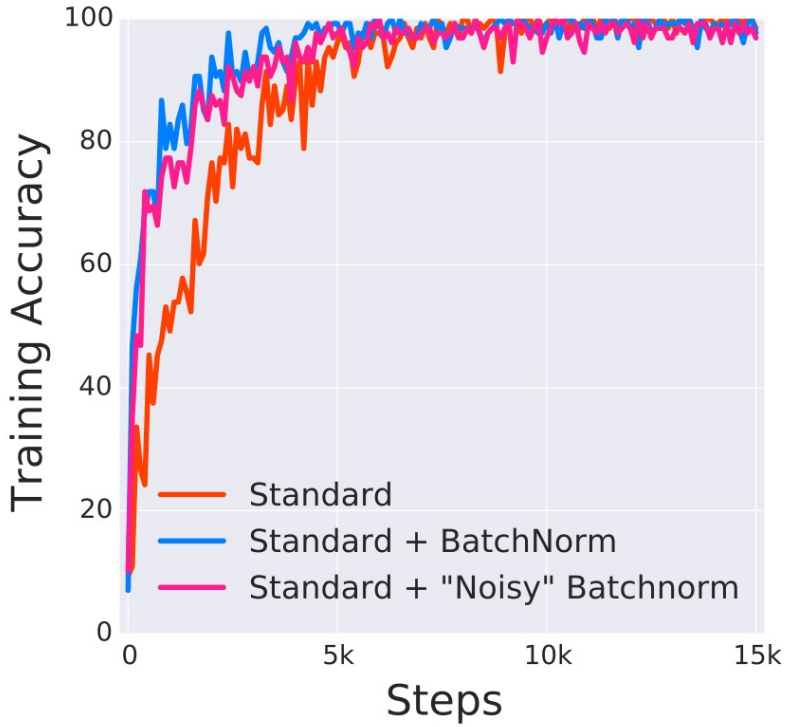
Andrew Ilyas* MIT ailyas@mit.edu

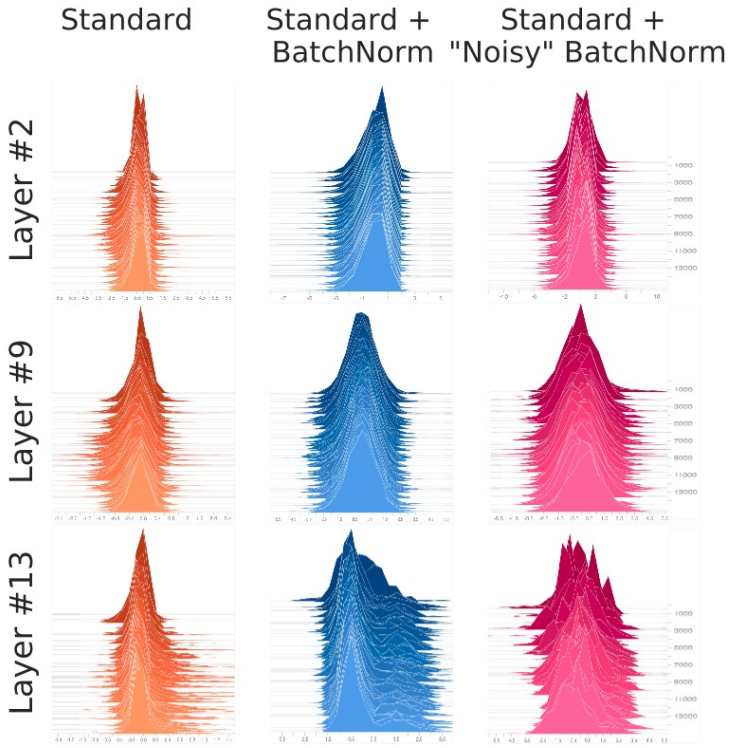
Aleksander Mądry MIT madry@mit.edu

https://arxiv.org/pdf/1805.11604.pdf

В чём польза от BatchNorm?

- Добавим шум после нормализации — хуже не становится!

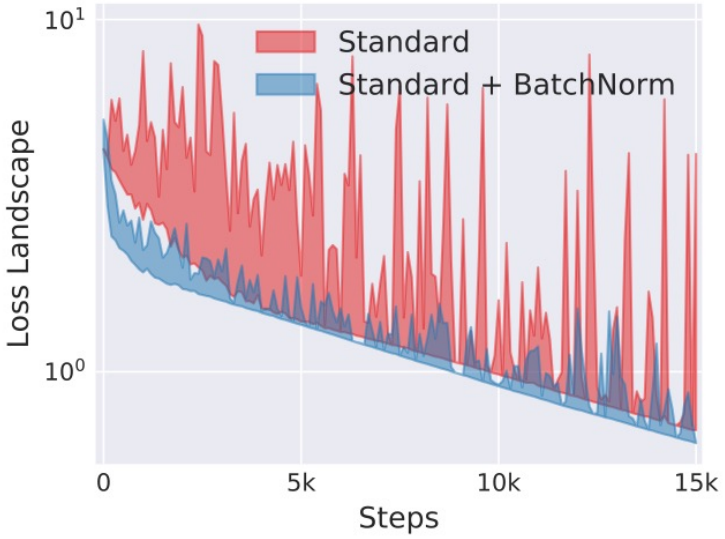




https://arxiv.org/pdf/1805.11604.pdf

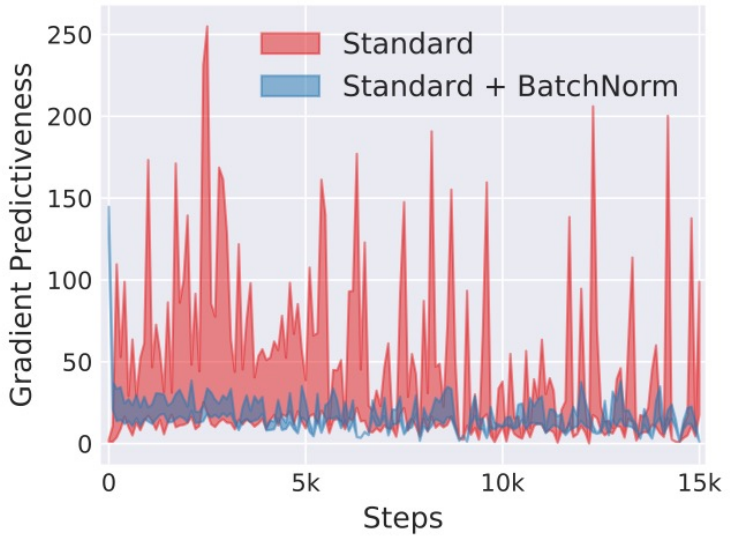
В чём польза от BatchNorm?

- Функционал ошибки становится более гладким!



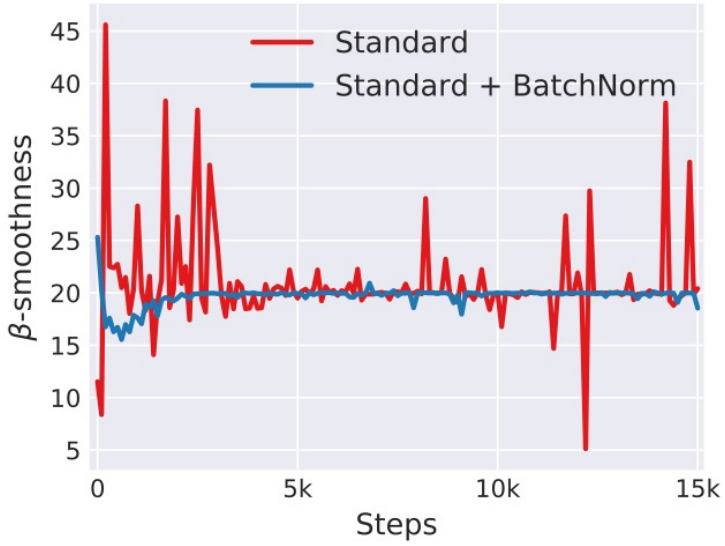
(a) loss landscape

(a) лосс при шаге вдоль градиента на разные длины шага: широкая полоса — изрезанный рельеф



(b) gradient predictiveness

(b) насколько при том же шаге меняется сам градиент



(c) "effective" β-smoothness

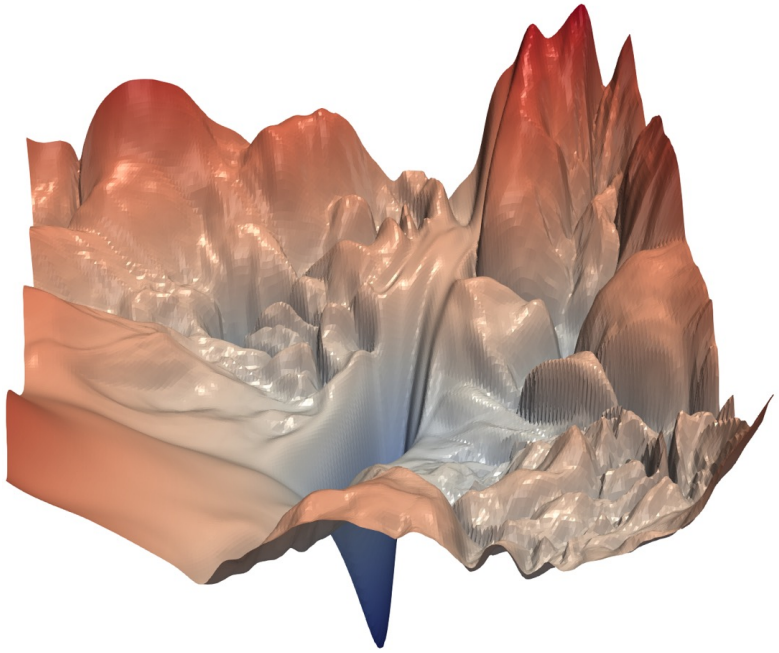
(c) максимум (b) на длину шага: как далеко можно шагнуть, не потеряв направление — отсюда больший learning rate

https://arxiv.org/pdf/1805.11604.pdf

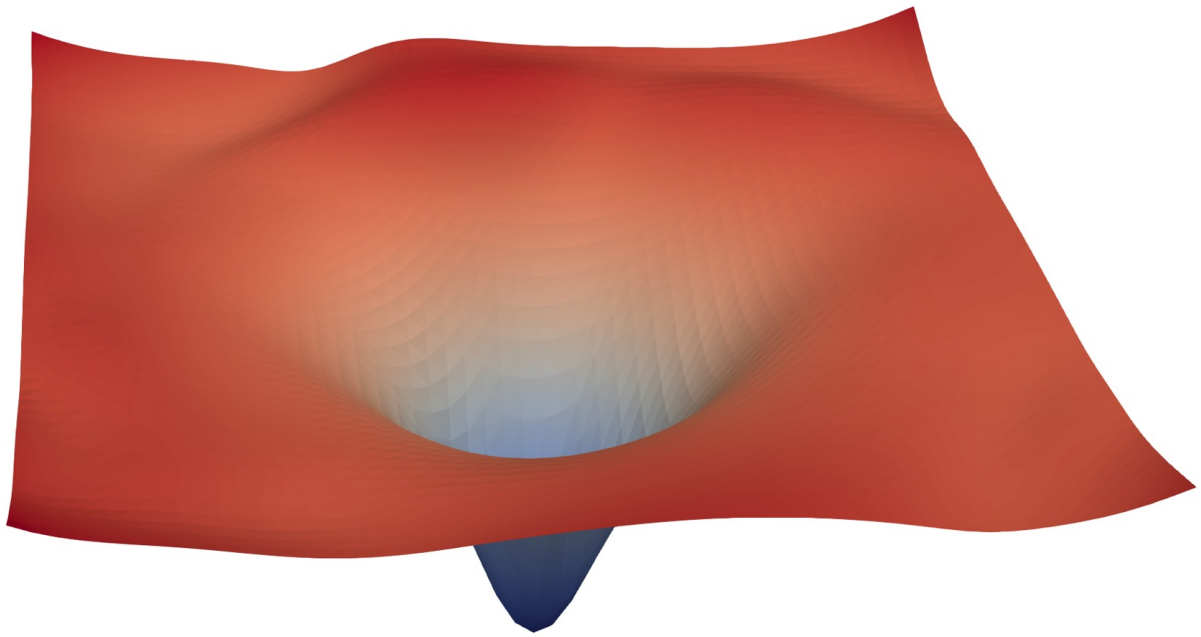
Ландшафт лосса и skip connections

- ResNet-56 на CIFAR-10, срез функционала ошибки по двум случайным направлениям в пространстве весов
- У всех сетей на картинках есть BatchNorm — без skip connections он не спасает
  - Li, Xu, Taylor, Studer, Goldstein. Visualizing the Loss Landscape of Neural Nets, NeurIPS 2018 — arxiv.org/abs/1712.09913

ResNet-56 без skip connections

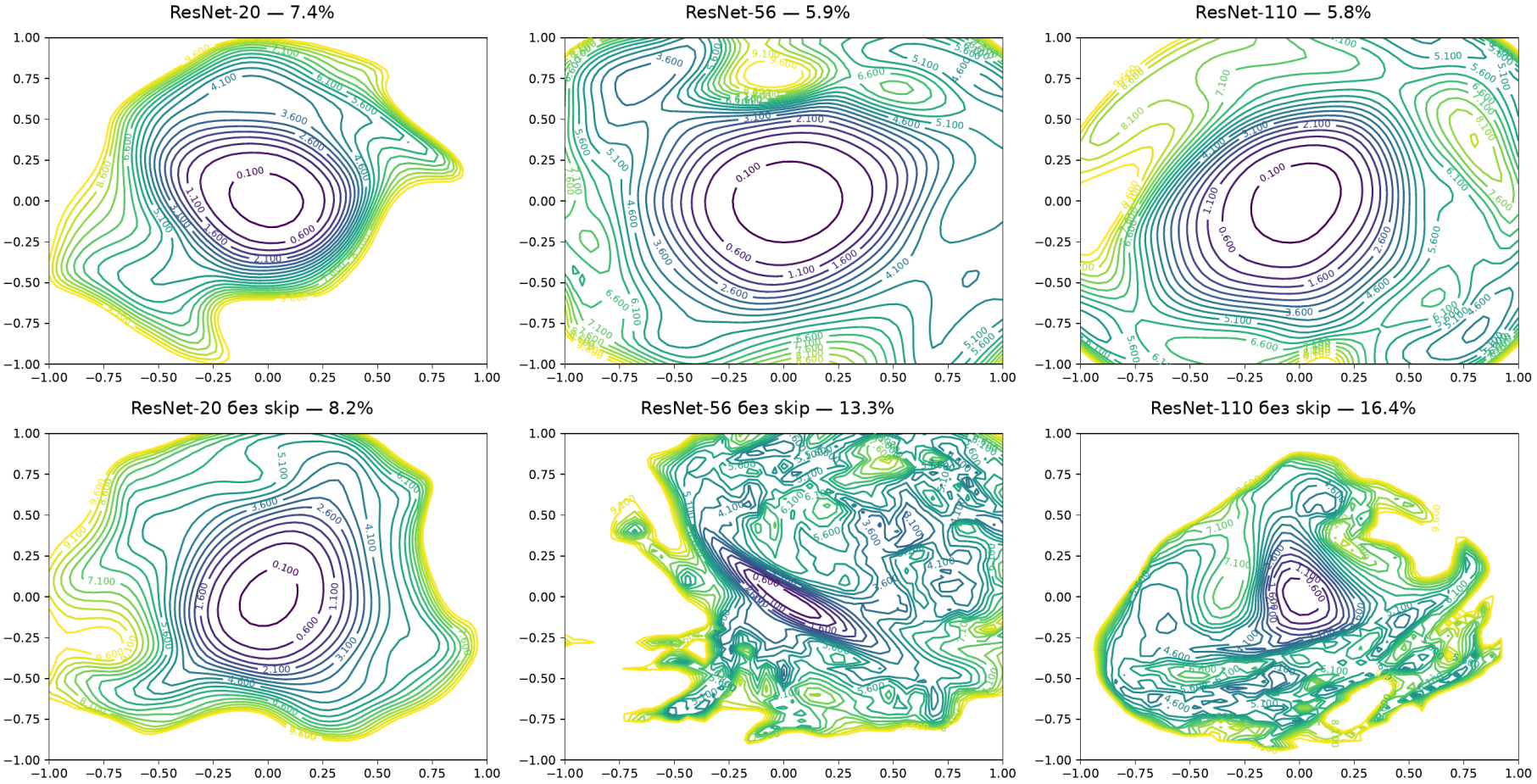


ResNet-56 со skip connections



Глубина против skip connections

- Те же срезы для ResNet-20 / 56 / 110; в подписи — ошибка на тесте
- Чем ровнее картинка, тем лучше обобщение
  - Li et al., 2018, Figure 5 (срезы с filter normalization)



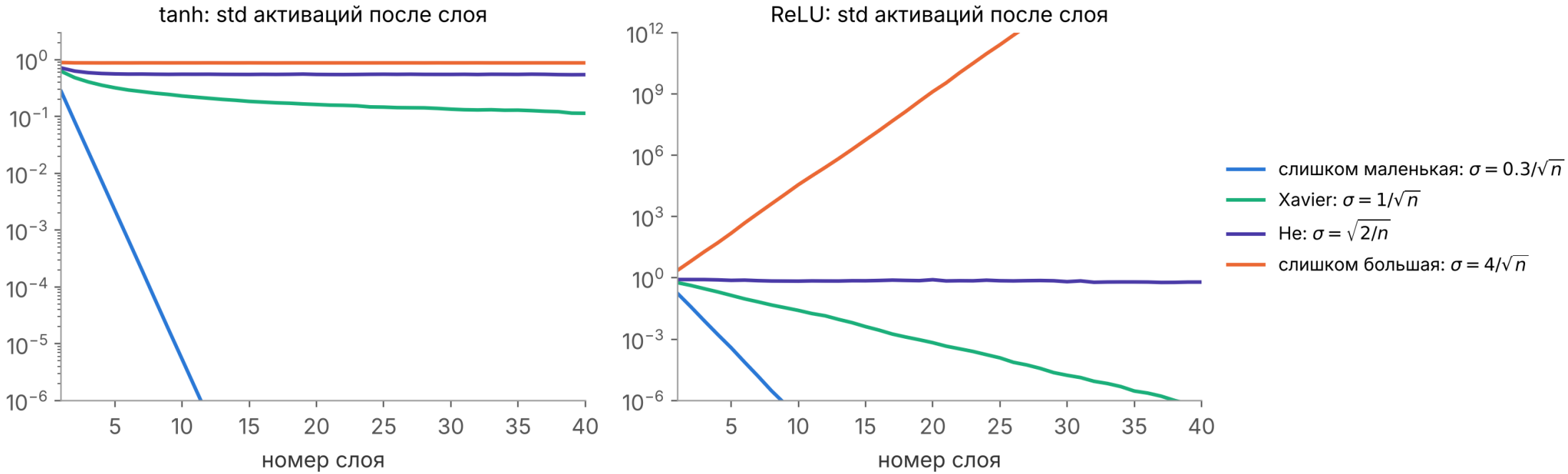
Проблемы BatchNorm

- Зависимость от размера батча: при батче 1–8 статистики шумные, при батче 1 дисперсия нулевая — нормировать нечего
- Разное поведение на обучении и применении: `model.train()` / `model.eval()`
  - забытый `eval()` — предсказания зависят от соседей по батчу
- Объекты в батче перестают быть независимыми — статистики связывают их
- В трансформерах длина последовательности переменная — батчевые статистики неудобны
- Выход: нормировать не по батчу, а внутри одного объекта → Layer Normalization

Инициализация

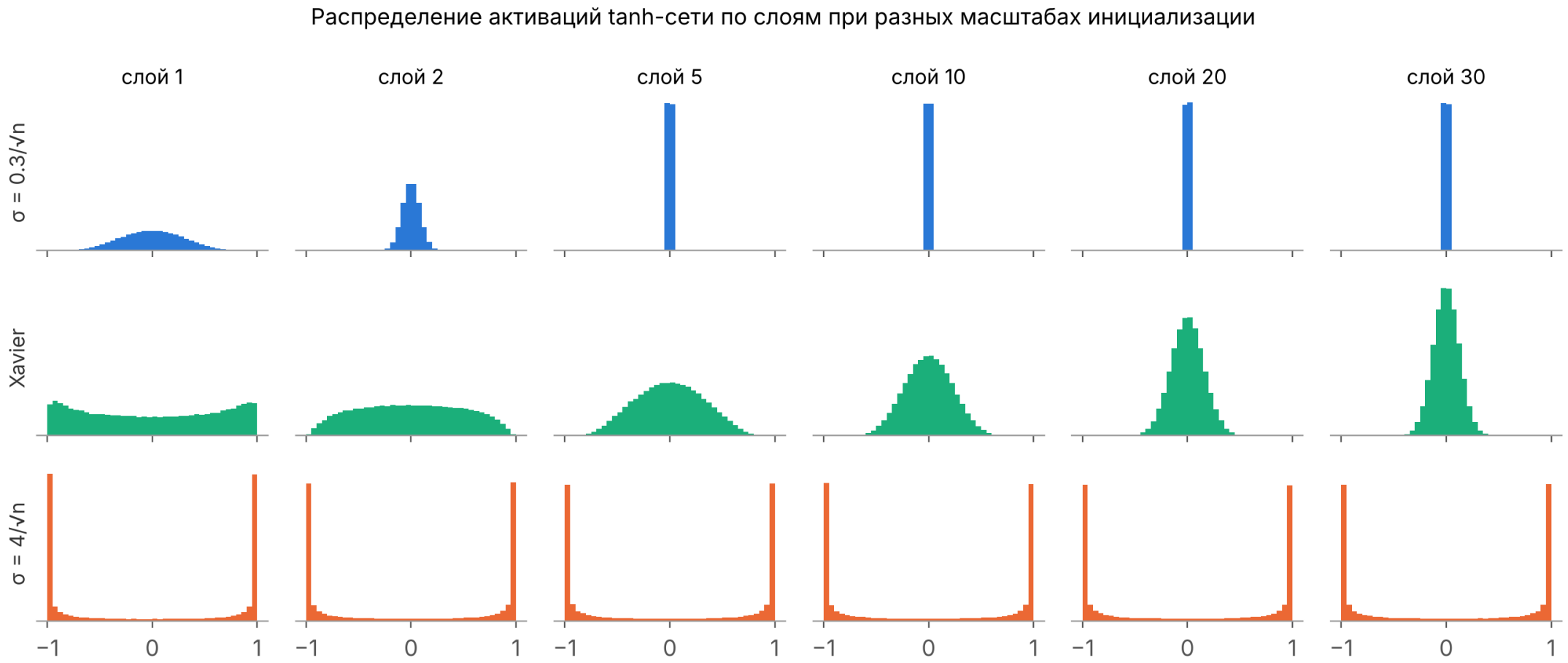
Зачем думать об инициализации

- Нулями или одним числом нельзя: все нейроны слоя получают одинаковые градиенты и навсегда остаются копиями друг друга
- Случайно тоже плохо
  - Маленькие веса — сигнал затухает с глубиной
  - Большие — tanh и сигмоида насыщаются, а с ReLU сигнал взрывается
- У обратного прохода те же проблемы
  - Градиент умножается на матрицу каждого слоя



Как это выглядит

- 30-слойная tanh-сеть ширины 256 на случайном входе: распределение активаций по слоям



Инициализация Xavier (2010)

- Линейный нейрон с n_{in} входами, веса и входы независимы и центрированы:

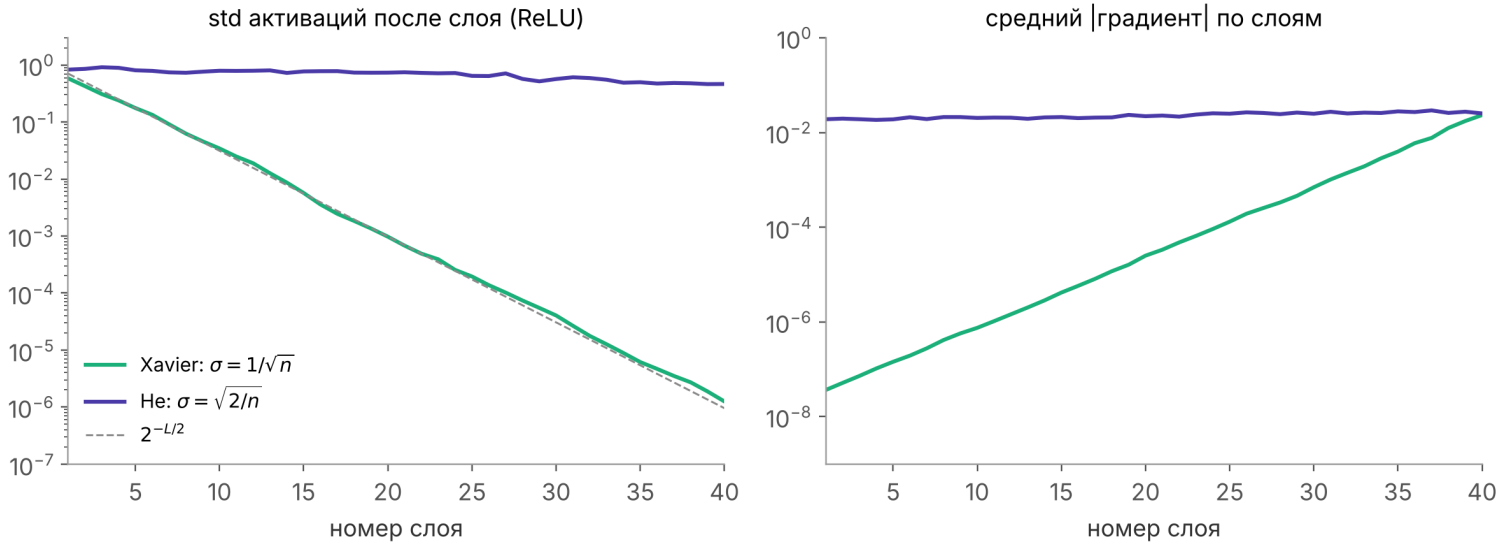
$$z = \sum_{i=1}^{n_{in}} w_i x_i\,, \qquad \mathbb{D}z = n_{in} \cdot \mathbb{D}w \cdot \mathbb{D}x$$

- Хотим сохранять масштаб сигнала, $\mathbb{D}z = \mathbb{D}x$: $\mathbb{D}w = \frac{1}{n_{in}}$; для градиентов то же с n_{out}
- Компромисс Glorot & Bengio: $\mathbb{D}w = \frac{2}{n_{in}+n_{out}}$
- Равномерный вариант: $U(-a, a)$, $a = \sqrt{\frac{6}{n_{in}+n_{out}}}$; для свёртки $n_{in} = k \cdot k \cdot C_{in}$
- Вывод сделан для линейной активации; для tanh около нуля это почти так же

https://proceedings.mlr.press/v9/glorot10a.html

Инициализация He (2015) для ReLU

- Слой с ReLU: $z = \sum_{i=1}^{n_{in}} w_i x_i$, $h = \mathrm{ReLU}(z) = \max(0, z)$ — вход следующего слоя
- z симметричен относительно нуля, половина значений обнуляется: $\mathbb{E}h^2 = \frac{1}{2}\mathbb{E}z^2 = \frac{1}{2}\mathbb{D}z$
- Следующий слой: $\mathbb{D}z' = n_{in} \cdot \mathbb{D}w' \cdot \mathbb{E}h^2 = n_{in} \cdot \mathbb{D}w' \cdot \frac{\mathbb{D}z}{2}$ → с Xavier масштаб падает вдвое на каждом слое, через 30 слоёв — в 2^{15} раз
- Поправка: $\mathbb{D}w = \frac{2}{n_{in}}$. Конкретно: $w \sim \mathrm{N}\left(0, \frac{2}{n_{in}}\right)$ или $\mathrm{U}(-a, a)$, $a = \sqrt{\frac{6}{n_{in}}}$; для свёртки $n_{in} = k \cdot k \cdot C_{in}$
- Из статьи: 22-слойная ReLU-сеть учится с обеими инициализациями, 30-слойная с Xavier не учится вовсе (градиенты затухают)



https://arxiv.org/abs/1502.01852

Инициализация на практике

- Принцип: масштаб сигнала и градиента не должен меняться от слоя к слою, формула зависит от нелинейности
- BatchNorm/LayerNorm сами приводят масштаб в порядок на каждом слое — поэтому вопрос стал менее острым
- Что делают сейчас:
  - берут значения по умолчанию из библиотеки (PyTorch: равномерная на $\pm 1/\sqrt{n_{in}}$ для Linear и Conv) и обычно не трогают — вместе с нормализацией этого хватает
  - в очень глубоких сетях следят, чтобы масштаб не рос с глубиной: последний слой каждого блока инициализируют маленьким или нулём, чтобы на старте сеть вела себя как мелкая
  - в больших моделях масштаб инициализации и длину шага согласуют с шириной сети, чтобы гиперпараметры, подобранные на маленькой модели, переносились на большую (µP, Yang et al., 2022)
  - чаще стартуют не со случайных весов, а с предобученной модели

https://arxiv.org/abs/2203.03466

Аугментация

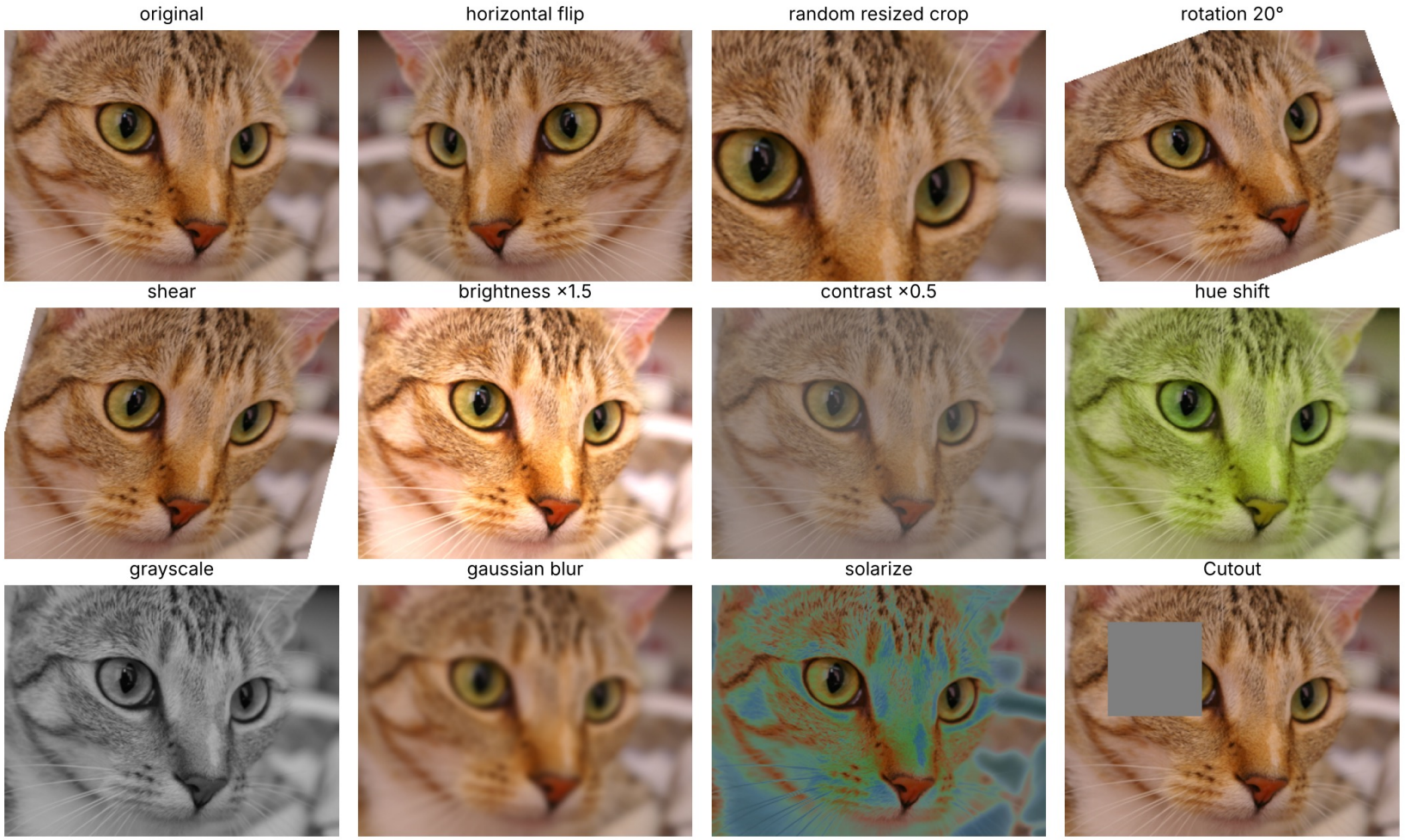
Аугментация: идея

- Мы заранее знаем, к чему модель должна быть устойчива: сдвиг, отражение, освещение, небольшой поворот
- Вряд ли стоит надеяться, что модель выучит это сама
- Синтезируем новые примеры небольшими возмущениями имеющихся — получим бесплатное расширение выборки
- По сути регуляризация: вшиваем в данные априорное знание об инвариантностях

- На практике — случайные преобразования к каждому батчу на лету
  - одна и та же картинка в каждой эпохе немного другая
- На валидации и тесте — без аугментаций (кроме test-time augmentation)

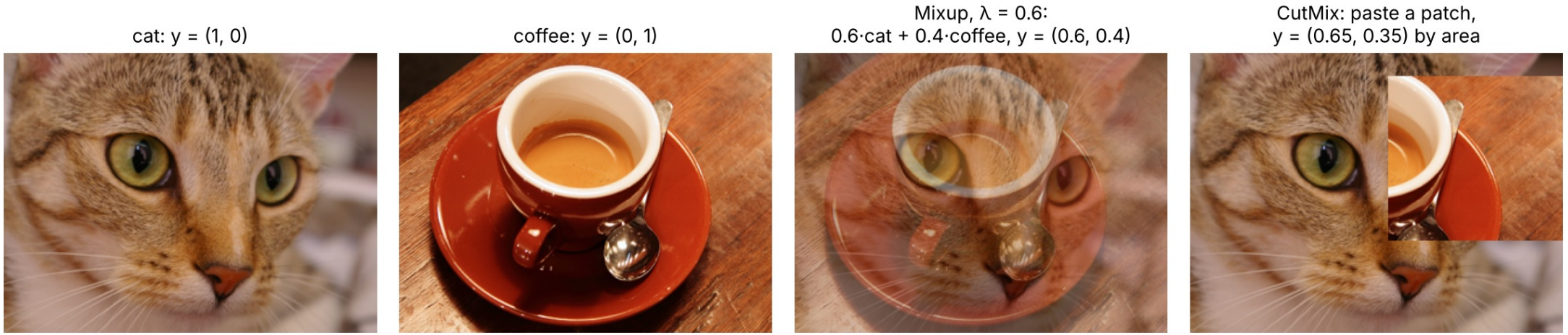
Примеры аугментаций

- Классический подход на ImageNet (ResNet, 2015) — случайный кроп с изменением масштаба и горизонтальное отражение
- Сегодня смешивают большее число аугментаций



Выбрасывание и смешивание

- Cutout / Random Erasing (2017): закрашиваем случайный прямоугольник — сеть не может полагаться на одну деталь
- Mixup (2018): $x = \lambda x_i + (1 - \lambda)x_j$, метки смешиваются так же; λ ~ Beta(α, α)
- CutMix (2019): вставляем кусок одной картинки в другую, метки — пропорционально площади



https://arxiv.org/abs/1710.09412 · https://arxiv.org/abs/1905.04899

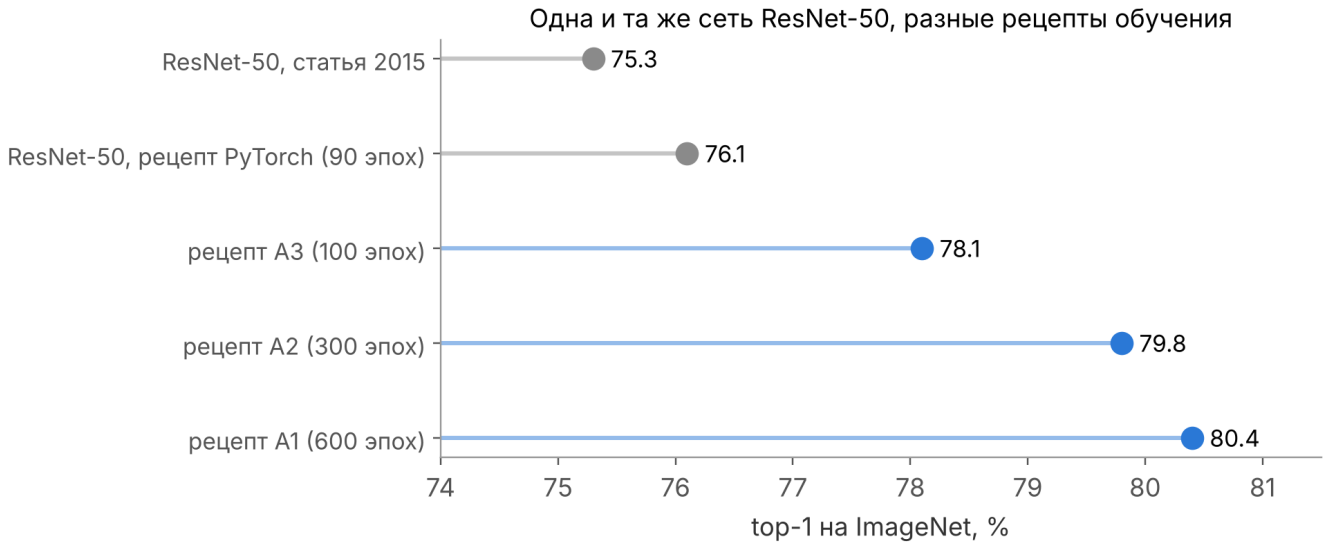
Автоматический подбор политики

- AutoAugment (2018): политику (какие операции, с какой вероятностью и силой) ищут обучением с подкреплением; тысячи GPU-часов на поиск
  - политика подбирается под датасет (поиск идёт на его уменьшенной копии с маленькой сетью)
  - сила — насколько существенно аугментация меняет картинку (градус поворота, степень изменения яркости и т.д.), в единой шкале
- RandAugment (2019): всего два гиперпараметра — число операций N и общая сила M; операции берутся случайно. На ImageNet ResNet-50: 76.3 → 77.6, столько же, сколько у AutoAugment
- TrivialAugment (2021): одна случайная операция со случайной силой, гиперпараметров нет — не хуже
- Вывод: раз выученная политика всё равно случайна и не зависит от картинки, простой случайный выбор преобразований работает не хуже

https://arxiv.org/abs/1909.13719 · https://arxiv.org/abs/2103.10158

Насколько это важно

- Рецепт обучения — оптимизатор и его гиперпараметры, расписание шага, число эпох, размер батча, аугментации, регуляризация, функция потерь, разрешение картинок, …
- Та же сеть ResNet-50, другой рецепт приводят к росту с 75.3% до 80.4% top-1 на ImageNet
- Сильные аугментации требуют долгого обучения, лучший рецепт — 600 эпох вместо 90
- Рецепт часто даёт больше, чем новая архитектура — сравнивать архитектуры честно можно только при одинаковом рецепте



Общее для A1–A3: LAMB + cosine, батч 2048, RandAugment, Mixup, CutMix, BCE-лосс
Отличия: 600 / 300 / 100 эпох; сила RandAugment 7 / 7 / 6; Mixup α 0.2 / 0.1 / 0.1; разрешение 224 / 224 / 160; repeated augmentation и stochastic depth — в A1 и A2; label smoothing — только в A1

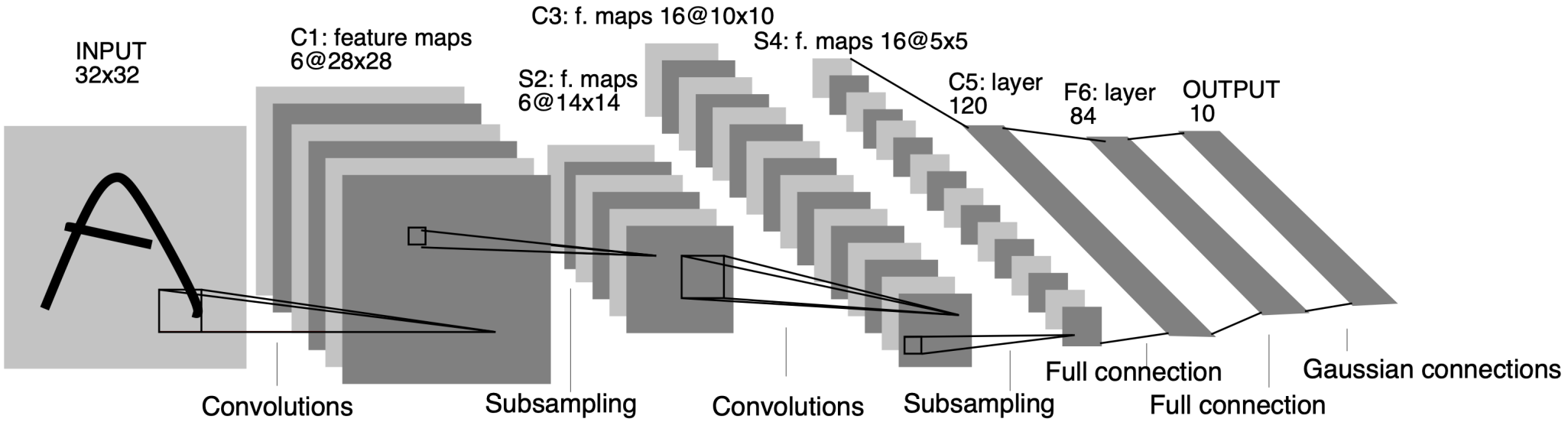
https://arxiv.org/abs/2110.00476

Оговорки и варианты

- Преобразование не должно менять метку: отражение цифр 6 и 9, поворот текста — нельзя; для спутниковых и медицинских снимков повороты, наоборот, естественны
- Test-time augmentation: на тесте несколько аугментаций одной картинки, предсказания усредняются (стандартный приём на соревнованиях)
- Другие модальности: текст — обратный перевод и замена синонимов; аудио — SpecAugment, маскирование полос спектрограммы
- Аугментация лишь добавляет инвариантности и не заменяет обогащение выборки

Архитектуры свёрточных сетей

LeNet (1998)



http://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf

LeNet (1998)

- Для данных MNIST
- Идея end-to-end обучения
- Использовали аугментацию
- Около 60.000 параметров
- Доля ошибок на тесте 0.8%

http://yann.lecun.com/exdb/publis/pdf/lecun-98.pdf

ImageNet



- ImageNet Large Scale Visual Recognition Challenge (ILSVRC), 2010-2017
- Классы — синсеты WordNet (с иерархией: собака → лабрадор)
- Полный ImageNet — 14 млн картинок на 21 тыс. классов
- ILSVRC:
  - 1000 классов (нет вложенных)
  - 1.28 млн обучающих изображений (732–1300 на класс), 50 тысяч валидационных, 100 тысяч тестовых
- Картинки собраны поисковиками по названиям классов, метки проверены разметчиками на Mechanical Turk
- Метрика — top-5 ошибка
  - ответ верный, если правильный класс среди пяти лучших гипотез модели
- Ошибка человека (одного тренированного разметчика) — около 5.1%

http://image-net.org/challenges/LSVRC/

AlexNet (2012)

ImageNet Classification with Deep Convolutional Neural Networks

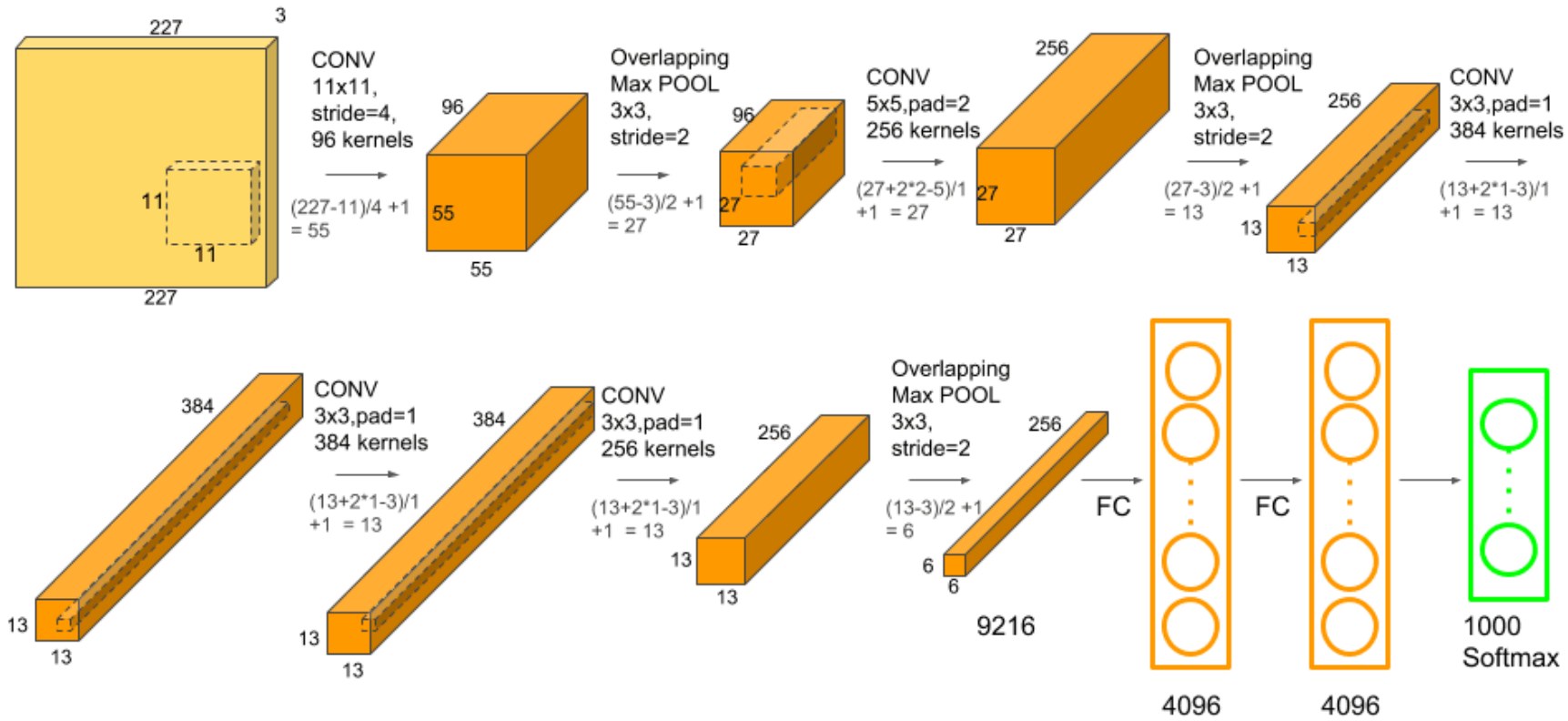
Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf

AlexNet (2012)



https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf

AlexNet (2012)

- 5 свёрточных слоёв + 3 полносвязных, первая свёртка 11×11 с шагом 4
- Используют ReLU, аугментацию, dropout
- Градиентный спуск с инерцией (momentum)
- Обучение на двух GPU (5–6 суток)
- Около 60 миллионов параметров
- Top-5 ошибка 15.3% против 26.2% у второго места

https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf

VGG (2014)

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* **& Andrew Zisserman+**
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen,az}@robots.ox.ac.uk

https://arxiv.org/pdf/1409.1556.pdf

VGG (2014)

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

VGG (2014)

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

https://arxiv.org/pdf/1409.1556.pdf

VGG (2014)

- Только маленькие свёртки 3×3
  - две подряд покрывают поле 5×5, три — 7×7, но параметров меньше
  - два слоя: $2 \cdot 9C^2 = 18C^2$ против $25C^2$ (при C каналах на входе и выходе каждого слоя)
  - три слоя: $3 \cdot 9C^2 = 27C^2$ против $49C^2$
  - +больше нелинейностей между ними
- Сеть собрана из одинаковых блоков по простому правилу (несколько 3×3 + pool, каналы ×2 после каждого pool: 64 → 128 → 256 → 512) — первая архитектура-конструктор, все следующие строятся аналогично
- Градиентный спуск с инерцией, dropout для двух первых полносвязных слоёв
- Хитрая инициализация: сначала обучается вариант A со случайными весами, потом им инициализируются более глубокие сети

https://arxiv.org/pdf/1409.1556.pdf

VGG (2014)

Table 3: **ConvNet performance at a single test scale.**

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
A	256	256	29.6	10.4
A-LRN	256	256	29.7	10.5
B	256	256	28.7	9.9
C	256	256	28.1	9.4
	384	384	28.1	9.3
	[256;512]	384	27.3	8.8
D	256	256	27.0	8.8
	384	384	26.8	8.7
	[256;512]	384	25.6	8.1
E	256	256	27.3	9.0
	384	384	26.9	8.7
	[256;512]	384	**25.5**	**8.0**

https://arxiv.org/pdf/1409.1556.pdf

VGG-16: параметры и память

- 138 млн параметров, из них 89% — в трёх полносвязных слоях
- Память под активации почти вся уходит на первые свёрточные слои
- Вывод: убрать полносвязные слои и сжимать каналы дешёвыми свёртками

